

INTRODUCTION

This documentation describes the functionality of the Dynamic Link Library: **MB4_1SL_interface.dll**. DLLs provide the standard benefits of shared libraries, such as modularity. Modularity allows changes to be made to code and data in a single self-contained DLL shared by several applications without any change to the applications themselves. Another benefit of the modularity is the use of generic interfaces for plug-ins. A single interface may be developed which allows old as well as new modules to be integrated seamlessly at run-time into pre-existing applications, without any modification to the application itself. Figure 1 shows the hierarchy of the layer model and illustrates the interconnectivity of the particular grades.

Features

- Reducing evaluation and design-in time and development costs
- Access to the configuration parameters of iC-MB4
- Transfer of predefined configuration HEX files
- Generation of EEPROM HEX files for device configuration
- Reading of iC-MB4 sensor data

Programming Environments

The DLL can be used in combination with the following programming languages.

C / C++ / LabVIEW / VBA

Other programming languages are not verified but may be capable to use this DLL.

Supported operating systems

MS Windows 7 / 8 / 10

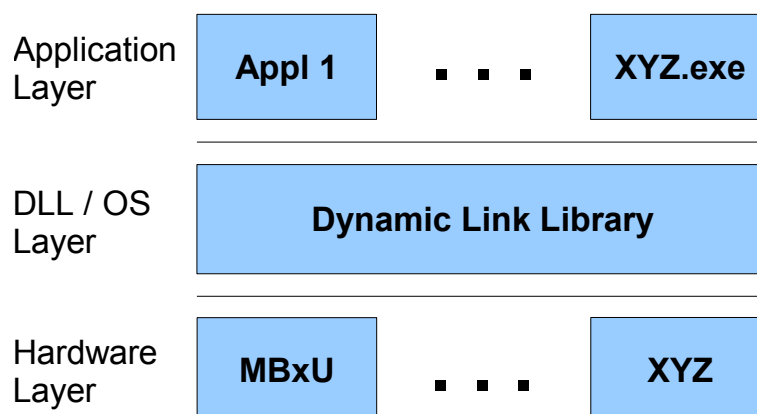


Figure 1: Three layer model

TABLE OF CONTENTS

INTRODUCTION	Page 1
FUNCTIONS	Page 4
MB4_Open	Page 4
MB4_Close	Page 4
MB4_SetInterface	Page 4
MB4_GetInterface	Page 5
MB4_SetConfig	Page 5
MB4_GetConfig	Page 6
MB4_SetParam	Page 6
MB4_GetParam	Page 7
MB4_WriteParams	Page 8
MB4_ReadParams	Page 8
MB4_ReadStatus	Page 8
MB4_WriteCommandRegister	Page 9
MB4_ReadSCDATA	Page 9
MB4_ReadRDATA	Page 9
MB4_WriteRDATA	Page 10
MB4_SaveParams	Page 10
MB4_LoadParams	Page 10
MB4_GetDLLVersion	Page 11
OPTIONAL FUNCTIONS	Page 12
MB4_SetRegister	Page 12
MB4_GetRegister	Page 12
MB4_WriteRegister	Page 13
MB4_ReadRegister	Page 13
MB4_SaveRegister	Page 14
MB4_LoadRegister	Page 14

iC-MB4 DLL

LIBRARY DESCRIPTION

preliminary



Rev A3, Page 3/21

PARAMETER AND ERROR CODING	Page 16
iC-MB4 Configuration Parameters	Page 18
DLL Configuration Parameters	Page 18
Error Definitions	Page 18
REVISION HISTORY	Page 19
APPENDIX	Page 19
Data Type Definitions	Page 19

FUNCTIONS

MB4_Open

Loads the library and returns a handle that will be used for subsequent accesses.

unsigned long **MB4_Open** (unsigned long *pulMB4Handle)

Parameters :

pulMB4Handle Pointer to the handle.

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

MB4_Close

Releases the handle and terminates established connections.

unsigned long **MB4_Close** (unsigned long ulMB4Handle)

Parameters :

ulMB4Handle Handle of the library.

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

MB4_SetInterface

Connects the interface defined by the parameter ulInterface.

unsigned long **MB4_SetInterface** (unsigned long ulMB4Handle,
unsigned long ulInterface)
char *pcInterfaceOption)

Parameters :

ulMB4Handle
ulInterface

Handle of the library.
Selection of the interface type.

Value	Name	Description
0	eMB4_NO_INTERFACE	no device
1	eMB4_MB4_SPI	SPI via MB3U (MB4_1D)
2	eMB4_MB3U	MB3U assembled with MB4

Table 1: Supported devices (MB4_InterfaceEnum)

pcInterfaceOption

reserved

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

MB4_GetInterface

Returns the selected interface from the DLL.

unsigned long **MB4_GetInterface** (unsigned long ulMB4Handle,
unsigned long *pulInterface)

Parameters :

ulMB4Handle
pulInterface

Handle of the library.
Pointer to the value of the interface type (see table 1, page 5).

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

MB4_SetConfig

Sets the DLL properties.

unsigned long **MB4_SetConfig** (unsigned long ulMB4Handle,
unsigned long ulConfig,
unsigned long ulValue)

Parameters :

ulMB4Handle
ulConfig
ulValue

Handle of the library.
The DLL configuration parameter. The list of parameters can be found in table 5, page 18.
The value of the DLL configuration parameter.

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

MB4_GetConfig

Gets the DLL properties.

unsigned long **MB4_GetConfig** (unsigned long ulMB4Handle,
unsigned long ulConfig,
unsigned long *pulValue)

Parameters :

ulMB4Handle	Handle of the library.
ulConfig	The DLL configuration parameter. The list of parameters can be found in table 5, page 18.
pulValue	Pointer to the value of the DLL configuration parameter.

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

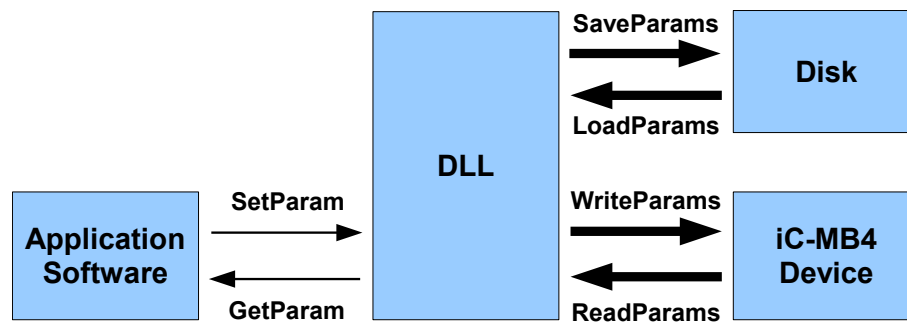


Figure 2: Illustration of the main functions of the iC-MB4 DLL.

MB4_SetParam

Sets an iC-MB4 parameter in the DLL with optional write. Ensure that the DLL memory and the iC register content are equal by using the **MB4_ReadParams** function before setting and writing new parameter values.

unsigned long **MB4_SetParam** (unsigned long ulMB4Handle,
unsigned long ulParam,
unsigned long ulValue,
unsigned long ulWriteVerify)

Parameters :

iC-MB4 DLL

LIBRARY DESCRIPTION



Rev A3, Page 7/21

ulMB4Handle
ulParam

ulValue
ulWriteVerify

Handle of the library.
The iC-MB4 configuration parameter. The list of parameters can be found in table 4, page 18.
The value of the iC-MB4 configuration parameter.

Value	Name	Description
0	eMB4_SETONLY	Sets the parameter to the DLL only
1	eMB4_WRITE	Sets the parameter to the DLL and writes the corresponding register to the iC-MB4 immediately.
2	eMB4_VERIFY	Sets the parameter to the DLL, writes the corresponding register to the iC-MB4 immediately and reads the register from the iC-MB4 for verification. The return value is an error code if verification fails.

Table 2: Write Verification Commands (MB4_WriteVerifyEnum)

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

Hints :

Configure parameters with calls of the function **MB4_SetParam**. Then use **MB4_WriteParams** to write parameters to the iC-MB4. See figure 2.

MB4_GetParam

Gets an iC-MB4 parameter from the DLL.

unsigned long **MB4_GetParam** (unsigned long ulMB4Handle,
unsigned long ulParam,
unsigned long *pulValue)

Parameters :

ulMB4Handle
ulParam

pulValue
Handle of the library.
The iC-MB4 configuration parameter. The list of parameters can be found in table 4, page 18.
Pointer to the value of the iC-MB4 configuration parameter.

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

Hints :

To get the values saved in the iC-MB4 registers use **MB4_ReadParams** before calling **MB4_GetParam**. See figure 2.

MB4_WriteParams

Writes parameters from the DLL to the iC-MB4 registers (address range = 0xC0.. 0xEF and Adr. 0xF5).

unsigned long **MB4_WriteParams** (unsigned long ulMB4Handle,
unsigned long ulVerify,
unsigned long *pulValid)

Parameters :

ulMB4Handle	Handle of the library.
ulVerify	Set to '1' reads registers for verification. Set to '0' writes without verification.
pulValid	Pointer to an array with a size of the count of registers (0x00.. 0xFF = 256). In case of verification, this array is used to specify which registers are valid. If the written value is equal to the read value, the function writes a '1' in the array at the position corresponding to the register. If the verification fails the array contains a '0' at this position.

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

MB4_ReadParams

Reads registers from the iC-MB4 to the DLL (address range = 0x00.. 0xFF).

unsigned long **MB4_ReadParams** (unsigned long ulMB4Handle)

Parameters :

ulMB4Handle	Handle of the library.
-------------	------------------------

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

MB4_ReadStatus

Reads the status register (Adr. 0xF0..0xF3) from the iC-MB4 to the DLL.

unsigned long **MB4_ReadStatus** (unsigned long ulMB4Handle,
unsigned long *pulStatus)

Parameters :

ulMB4Handle	Handle of the library.
pulStatus	Pointer to an array with a size of 4. The array contains the values of the status registers (Adr. 0xF0..0xF3).

Parameters :

<code>ulMB4Handle</code>	Handle of the library.
<code>ulByteCount</code>	Number of registers.
<code>pulData</code>	Pointer to an array with a size of <code>ulByteCount</code> .

Return Value :

`eMB4_OK` if successful, otherwise the return value is an error code (see table 6, page 18).

MB4_WriteRDATA

Writes RDATA registers to iC-MB4.

unsigned long MB4_WriteRDATA	(unsigned long <code>ulMB4Handle</code> , unsigned long <code>ulByteCount</code> , unsigned long <code>*pulData</code>)
-------------------------------------	---

Parameters :

<code>ulMB4Handle</code>	Handle of the library.
<code>ulByteCount</code>	Number of registers.
<code>pulData</code>	Pointer to an array with a size of <code>ulByteCount</code> .

Return Value :

`eMB4_OK` if successful, otherwise the return value is an error code (see table 6, page 18).

MB4_SaveParams

Saves iC-MB4 parameters from the DLL to a specified file. The file format is Intel HEX.

unsigned long MB4_SaveParams	(unsigned long <code>ulMB4Handle</code> , char <code>*pcFilename</code>)
-------------------------------------	---

Parameters :

<code>ulMB4Handle</code>	Handle of the library.
<code>pcFilename</code>	Absolute path of the file.

Return Value :

`eMB4_OK` if successful, otherwise the return value is an error code (see table 6, page 18).

MB4_LoadParams

Loads iC-MB4 parameters from the specified file to the DLL. The file format is Intel HEX.

unsigned long **MB4_LoadParams** (unsigned long ulMB4Handle,
char *pcFilename)

Parameters :

ulMB4Handle	Handle of the library.
pcFilename	Absolute path of the file.

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

MB4_GetDLLVersion

Gets the version number of the DLL.

unsigned long **MB4_GetDLLVersion** (unsigned long *pulDllVersion))

Parameters :

*pulDllVersion	Pointer to an array with two elements that contains the DLL Version.
----------------	--

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

Hints :

Index '1' of the version number comprises the major version number, index '0' the minor version number. The function does not need a handle.

OPTIONAL FUNCTIONS

MB4_SetRegister

Sets an iC-MB4 register in the DLL with optional write.

unsigned long **MB4_SetRegister** (unsigned long ulMB4Handle,
unsigned long ulAdr,
unsigned long ulValue,
unsigned long ulWriteVerify)

Parameters :

ulMB4Handle
ulAdr
ulValue
ulWriteVerify

Handle of the library.
The address of the register.
The value of the register.

Value	Name	Description
0	eMB4_SETONLY	Sets the parameter to the DLL only
1	eMB4_WRITE	Sets the parameter to the DLL and writes the corresponding register to the iC-MB4 immediately.
2	eMB4_VERIFY	Sets the parameter to the DLL, writes the corresponding register to the iC-MB4 immediately and reads the register from the iC-MB4 for verification. The return value is an error code if verification fails.

Table 3: Write Verification Commands (MB4_WriteVerifyEnum)

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

Hints :

This function writes a whole register to the DLL instead of a single parameter. Use function **MB4_WriteParams** to write these changes to the iC-MB4 RAM.

MB4_GetRegister

Gets an iC-MB4 register from the DLL.

unsigned long **MB4_GetRegister** (unsigned long ulMB4Handle,
unsigned long ulAdr,
unsigned long *pulValue)

Parameters :

ulMB4Handle	Handle of the library.
ulAdr	The address of the register.
pulValue	Pointer to the value of the register.

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

Hints :

To get the register value saved in the iC-MB4 RAM use **MB4_ReadParams** before calling **MB4_GetRegister**.

MB4_WriteRegister

Writes data from an array to the iC-MB4 registers (address range = start address + number of bytes - 1).

unsigned long MB4_WriteRegister	(unsigned long ulMB4Handle, unsigned long ulAdr, unsigned long *pulNumberOfBytes, unsigned long *pulData)
--	---

Parameters :

ulMB4Handle	Handle of the library.
ulAdr	The address of the register.
pulNumberOfBytes	Pointer to the number of registers. After execution, this parameter contains the number of successfully written bytes.
pulData	Pointer to an array containing the values.

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

MB4_ReadRegister

Reads data from the iC-MB4 registers to an array (address range = start address + number of bytes - 1).

unsigned long MB4_ReadRegister	(unsigned long ulMB4Handle, unsigned long ulAdr, unsigned long *pulNumberOfBytes, unsigned long *pulData)
---------------------------------------	---

Parameters :

ulMB4Handle	Handle of the library.
ulAdr	The address of the register.
pulNumberOfBytes	Pointer to the number of registers.
pulData	Pointer to an array containing the values.

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

MB4_SaveRegister

Saves data from an array to a file (address range = start address + number of bytes -1). The file format is Intel HEX.

unsigned long **MB4_SaveRegister** (unsigned long ulMB4Handle,
char *pcFilename,
unsigned long ulAdr,
unsigned long *pulNumberOfBytes,
unsigned long *pulData)

Parameters :

ulMB4Handle	Handle of the library.
pcFilename	Absolute path of the file.
ulAdr	Address of the first register.
pulNumberOfBytes	Pointer to the number of registers to save.
pulData	Pointer to an array containing the values.

Return Value :

eMB4_OK if successful, otherwise the return value is an error code (see table 6, page 18).

Hints :

If the file already exists, only the data of the specified address range will be replaced.

MB4_LoadRegister

Loads data from a file to an array (address range = start address + number of bytes -1). The file format is Intel HEX.

unsigned long **MB4_LoadRegister** (unsigned long ulMB4Handle,
char *pcFilename,
unsigned long ulAdr,
unsigned long *pulNumberOfBytes,
unsigned long *pulData)

Parameters :

<code>ulMB4Handle</code>	Handle of the library.
<code>pcFilename</code>	Absolute path of the file.
<code>ulAdr</code>	Address of the first register.
<code>pulNumberOfBytes</code>	Pointer to the number of registers to load.
<code>pulData</code>	Pointer to an array containing the values.

Return Value :

`eMB4_OK` if successful, otherwise the return value is an error code (see table 6, page 18).

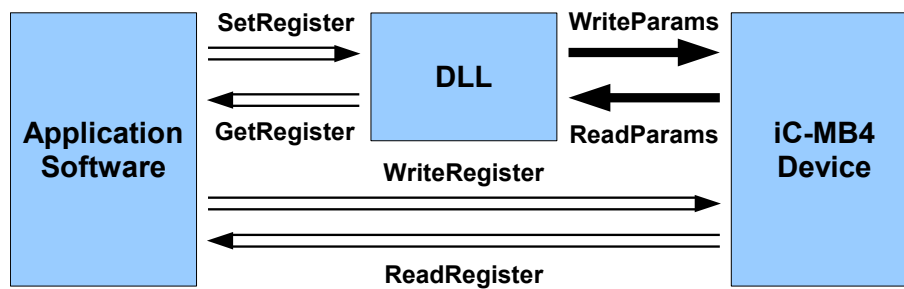


Figure 3: Illustration of the optional functions of the iC-MB4 DLL.

iC-MB4 DLL

LIBRARY DESCRIPTION

PARAMETER AND ERROR CODING

Value	Parameter	Description (see latest data sheet)
0	eMB4_SCDLEN1	
1	eMB4_ENSCD1	
2	eMB4_GRAYS1	
3	eMB4_SCRCPOLY1	
4	eMB4_SELRCRS1	
5	eMB4_SCRCSTART1	
6	eMB4_SCDLEN2	
7	eMB4_ENSCD2	
8	eMB4_GRAYS2	
9	eMB4_SCRCPOLY2	
10	eMB4_SELRCRS2	
11	eMB4_SCRCSTART2	
12	eMB4_SCDLEN3	
13	eMB4_ENSCD3	
14	eMB4_GRAYS3	
15	eMB4_SCRCPOLY3	
16	eMB4_SELRCRS3	
17	eMB4_SCRCSTART3	
18	eMB4_SCDLEN4	
19	eMB4_ENSCD4	
20	eMB4_GRAYS4	
21	eMB4_SCRCPOLY4	
22	eMB4_SELRCRS4	
23	eMB4_SCRCSTART4	
24	eMB4_SCDLEN5	
25	eMB4_ENSCD5	
26	eMB4_GRAYS5	
27	eMB4_SCRCPOLY5	
28	eMB4_SELRCRS5	
29	eMB4_SCRCSTART5	
30	eMB4_SCDLEN6	
31	eMB4_ENSCD6	
32	eMB4_GRAYS6	
33	eMB4_SCRCPOLY6	
34	eMB4_SELRCRS6	
35	eMB4_SCRCSTART6	
36	eMB4_SCDLEN7	
37	eMB4_ENSCD7	
38	eMB4_GRAYS7	
39	eMB4_SCRCPOLY7	
40	eMB4_SELRCRS7	
41	eMB4_SCRCSTART7	
42	eMB4_SCDLEN8	
43	eMB4_ENSCD8	
44	eMB4_GRAYS8	
45	eMB4_SCRCPOLY8	

iC-MB4 DLL

LIBRARY DESCRIPTION

Value	Parameter	Description (see latest data sheet)
46	eMB4_SELCRCS8	
47	eMB4_SCRCSTART8	
48	eMB4_REGADR	
49	eMB4_WNR	
50	eMB4_REGNUM	
51	eMB4_CHSEL	
52	eMB4_HOLDCDM	
53	eMB4_EN_MO	
54	eMB4_SLAVEID	
55	eMB4_REGVERS	
56	eMB4_CTS	
57	eMB4_FREQ	
58	eMB4_SINGLEBANK	
59	eMB4_NOCRC	
60	eMB4_FREQAGS	
61	eMB4_MO_BUSY	
62	eMB4_REVISION	
63	eMB4_VERSION	
64	eMB4_SLAVELOC5	
65	eMB4_CFGCH1	
66	eMB4_CFGCH2	
67	eMB4_ACTNSENS	
68	eMB4_EOT	
69	eMB4_REGEND	
70	eMB4_NREGERR	
71	eMB4_NSCDERR	
72	eMB4_NDELAYERR	
73	eMB4_NAGSERR	
74	eMB4_NERR	
75	eMB4_SVALID1	
76	eMB4_SVALID2	
77	eMB4_SVALID3	
78	eMB4_SVALID4	
79	eMB4_SVALID5	
80	eMB4_SVALID6	
81	eMB4_SVALID7	
82	eMB4_SVALID8	
83	eMB4_REGBYTES	
84	eMB4_CDSSEL	
85	eMB4_CDMTIMEOUT	
86	eMB4_AGS	
87	eMB4_INSTR	
88	eMB4_INIT	
89	eMB4_SWBANK	
90	eMB4_HOLDBANK	
91	eMB4_BREAK	
92	eMB4_CLKENI	
93	eMB4_ENTEST	
94	eMB4_CFGIF	

iC-MB4 DLL

LIBRARY DESCRIPTION

Value	Parameter	Description (see latest data sheet)
95	eMB4_MAFS	
96	eMB4_MAVS	
97	eMB4_MAF0	
98	eMB4_MAV0	
99	eMB4_SL1	
100	eMB4_CDS1	
101	eMB4_SL2	
102	eMB4_CDS2	
103	eMB4_SWBANKFAILS	
104	eMB4_IDS	
105	eMB4_IDA_TEST	
106	eMB4_CMD	

Table 4: iC-MB4 Configuration Parameters (MB4_ParamEnum)

Value	Parameter	Description
0	eMB4_FREQ_SPI	SPI frequency

Table 5: DLL Configuration Parameters (MB4_ConfigDataEnum)

Value	Error Message	Description (Solution)
0	eMB4_OK	Function call successful
1	eMB4_INVALID_HANDLE	Use the handle returned by the MB4_Open function
2	eMB4_INTERFACEDRIVER_NOT_FOUND	Install FTDI Interface Drivers
3	eMB4_INTERFACE_NOT_FOUND	Check wire connections and power supply
4	eMB4_INVALID_INTERFACE	Interface not specified. See table 1, page 5
5	eMB4_INVALID_PARAMETER	Parameter not specified. See table 4, page 18
6	eMB4_INVALID_ADDRESS	Address out of range
7	eMB4_INVALID_VALUE	Value out of range
8	eMB4_USB_ERROR	Check USB to interface connection
9	eMB4_FILE_NOT_FOUND	Check if file exists
10	eMB4_INVALID_FILE	Check file format
11	eMB4_VERIFY_FAILED	Check device connection
12	eMB4_NO_INTERFACE_SELECTED	Select an interface before using this function
13	eMB4_SPICOMM_FAILED	Check interface to iC-MB4 connection. Check power supply.
14	eMB4_FILE_ACCESS_DENIED	Insufficient permission to access file.

Table 6: Error Definitions (MB4_ErrorEnum)

iC-MB4 DLL

LIBRARY DESCRIPTION



Rev A3, Page 19/21

REVISION HISTORY

Modifications made to this document are listed below. For modifications made to the corresponding DLL see the latest release notes (included in the DLL zip package).

Rev	Notes	Pages affected	DLL reference
A1	Initial version		Version 1.0
A2	Chapter Revision History revised	19	Version 1.1
A3	MB4_WriteParams changed	8	Version 1.2

Table 7: Revision History

APPENDIX

Prefix	Type	Description
ul	unsigned long	Data type size of 32 bit; positive values only
pul	pointer of unsigned long	
c	char	Data type size of 8 bit
pc	pointer of char	Null-terminated string
d	double	Double precision floating point
pd	pointer of double	
e	enum	List of items

Table 8: Data Type Definitions

Application Example

Listing 1: Source code example (C++)

```
#include "MB4_1SL_interface.h"
#include <iostream>
using namespace std;
unsigned long ulMB4Handle;

int main()
{
    unsigned long ulaVersion[2];
    unsigned long ulError;

    //Get the version number of the DLL
    MB4_GetDLLVersion(ulaVersion);
    cout << "MB4_1SL_interface.dll_Version:_" << ulaVersion[1] << "." << ulaVersion[0] << endl << endl;

    //Load the library and get a handle that will be used for subsequent accesses.
    ulError = MB4_Open(&ulMB4Handle);

    //Connect the interface MB4_1D
    ulError = MB4_SetInterface(ulMB4Handle, eMB4_MB4_SPI, "");

    if (ulError)
    {
        switch (ulError){
            case eMB4_INTERFACEDRIVER_NOT_FOUND:
                cout << "MB4_SetInterface_failed_(eMB4_INTERFACEDRIVER_NOT_FOUND):" << endl
        }
    }
```

iC-MB4 DLL

LIBRARY DESCRIPTION



Rev A3, Page 20/21

```
<< "Install_FTDI_Interface_Drivers" << endl; break;
case eMB4_INTERFACE_NOT_FOUND:
    cout << "MB4_SetInterface_failed_(eMB4_INTERFACE_NOT_FOUND):_" << endl
    << "Check_wire_connections_and_power_supply" << endl; break;
default:
    cout << "MB4_SetInterface_failed_(errorcode:_" << ulError << ")"
    << endl; break;
}
}
else
{
    cout << "Interface_MB4_SPI_opened" << endl << endl;

    //Read registers from the iC-MB4 to the DLL
    ulError = MB4_ReadParams(ulMB4Handle);

    //Configure master
    ulError = MB4_SetParam(ulMB4Handle, eMB4_FREQ, 4, eMB4_WRITE);
    ulError = MB4_SetParam(ulMB4Handle, eMB4_CHSEL, 1, eMB4_WRITE);
    ulError = MB4_SetParam(ulMB4Handle, eMB4_CFGCH1, 1, eMB4_WRITE);
    ulError = MB4_SetParam(ulMB4Handle, eMB4_REGVERS, 1, eMB4_WRITE);
    ulError = MB4_SetParam(ulMB4Handle, eMB4_CTS, 1, eMB4_WRITE);

    //Set the properties of the connected slave(e.g. iC-NQC)
    ulError = MB4_SetParam(ulMB4Handle, eMB4_ENSCD1, 1, eMB4_WRITE);
    ulError = MB4_SetParam(ulMB4Handle, eMB4_SCDLEN1, 11, eMB4_WRITE); //Datalength: 12
    ulError = MB4_SetParam(ulMB4Handle, eMB4_SELCRCS1, 1, eMB4_WRITE);
    ulError = MB4_SetParam(ulMB4Handle, eMB4_SCRCPOLY1, 0x21, eMB4_WRITE); //Polynome: 0x43

    //Start BiSS transfer
    ulError = MB4_SetParam(ulMB4Handle, eMB4_INSTR, 2, eMB4_SETONLY);
    ulError = MB4_WriteCommandRegister(ulMB4Handle);

    //Read Status
    unsigned long ulaStatus[4];
    ulError = MB4_ReadStatus(ulMB4Handle, ulaStatus);
    unsigned long ulNerr;
    ulError = MB4_GetParam(ulMB4Handle, eMB4_NERR, &ulNerr);
    cout << "Status_Reg_0xF0:_0x" << hex << uppercase << ulaStatus[0] << endl;

    //Read SCDData
    unsigned long ulaData[8];
    if (ulNerr == 1)
        ulError = MB4_ReadSCDATA(ulMB4Handle, 1, ulaData);

    cout << "SCD:_0x" << hex << uppercase << ulaData[0] << "_0x" << ulaData[1] << endl;
}

//Release the handle and terminate established connections
MB4_Close(ulMB4Handle);
}
```

iC-MB4 DLL

LIBRARY DESCRIPTION



Rev A3, Page 21/21

iC-Haus expressly reserves the right to change its products and/or specifications. An Infoletter gives details as to any amendments and additions made to the relevant current specifications on our internet website www.ichaus.com/infoletter and is automatically generated and shall be sent to registered users by email. Copying – even as an excerpt – is only permitted with iC-Haus' approval in writing and precise reference to source.

The data specified is intended solely for the purpose of product description and shall represent the usual quality of the product. In case the specifications contain obvious mistakes e.g. in writing or calculation, iC-Haus reserves the right to correct the specification and no liability arises insofar that the specification was from a third party view obviously not reliable. There shall be no claims based on defects as to quality in cases of insignificant deviations from the specifications or in case of only minor impairment of usability.

No representations or warranties, either expressed or implied, of merchantability, fitness for a particular purpose or of any other nature are made hereunder with respect to information/specification or the products to which information refers and no guarantee with respect to compliance to the intended use is given. In particular, this also applies to the stated possible applications or areas of applications of the product.

iC-Haus products are not designed for and must not be used in connection with any applications where the failure of such products would reasonably be expected to result in significant personal injury or death (*Safety-Critical Applications*) without iC-Haus' specific written consent. Safety-Critical Applications include, without limitation, life support devices and systems. iC-Haus products are not designed nor intended for use in military or aerospace applications or environments or in automotive applications unless specifically designated for such use by iC-Haus.

iC-Haus conveys no patent, copyright, mask work right or other trade mark right to this product. iC-Haus assumes no liability for any patent and/or other trade mark rights of a third party resulting from processing or handling of the product and/or any other use of the product.

Software Disclaimer

Software and its documentation is provided by iC-Haus GmbH or contributors "AS IS" and is subject to the iC-Haus End User License Agreement (www.ichaus.com/EULA). Any express or implied warranties, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose are disclaimed. In no event shall iC-Haus GmbH or contributors be liable for any direct, indirect, incidental, special, exemplary, or consequential damages (including, but not limited to, procurement of substitute goods or services; loss of use, data, or profits; or business interruption) however caused and on any theory of liability, whether in contract, strict liability, or tort (including negligence or otherwise) arising in any way out of the use of iC-Haus GmbH software, even if advised of the possibility of such damage.